

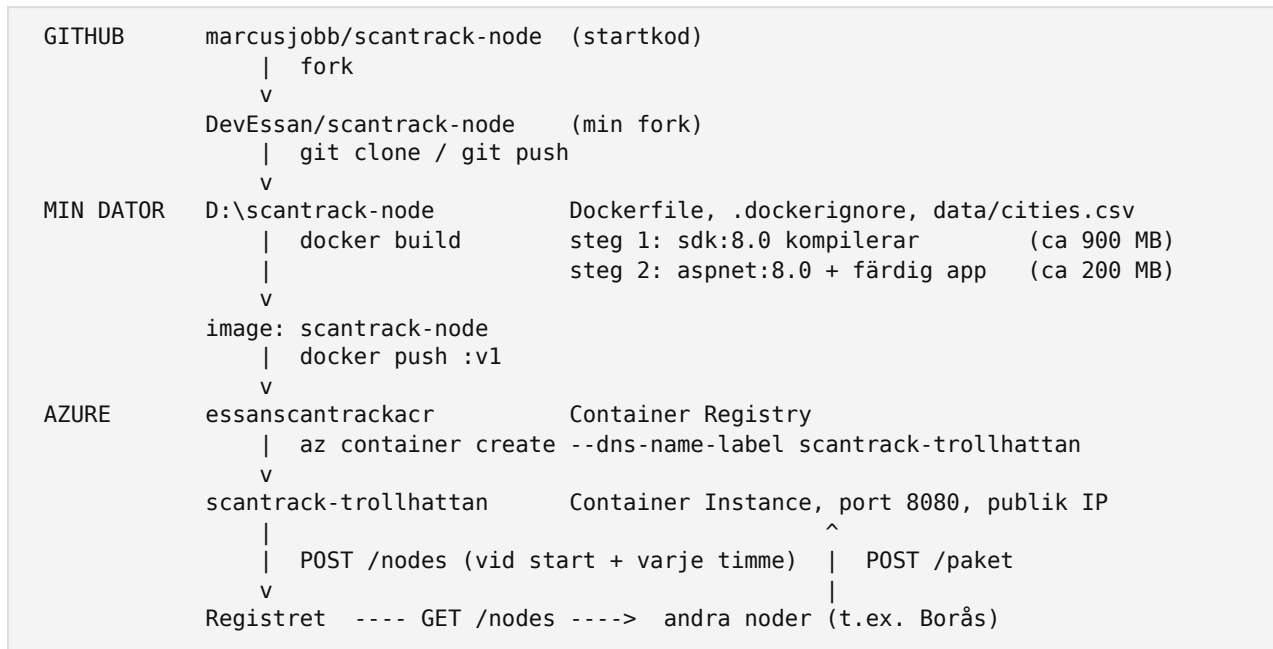
ScanTrack Node — Trollhättan

Essan Alshakerchi · Administrera molnlösningar, ITHS · 2026-09-06

Live-nod: <http://scantrack-trollhattan.swedencentral.azurecontainer.io:8080/status>

Repo: <https://github.com/DevEssan/scantrack-node> (fork av [marcusjobb/scantrack-node](#))

1. Arkitektur



Jag forkade startkoden till mitt GitHub-konto och klonade forken. I

PackageForwarder.ForwardAsync slår noden upp nästa stads adress i registret, serialiserar paketet och skickar det med POST /paket, med en timeout på tio sekunder.

HeartbeatService är en BackgroundService som registrerar noden på nytt var 60:e minut, och POST /forceheartbeat tvingar fram en registrering direkt, max en gång per tio minuter.

Dockerfilen är en multi-stage build. Steg 1 använder SDK-imagen: först kopieras bara .csproj och dotnet restore körs, sedan kopieras resten av koden och dotnet publish -c Release --no-restore körs. Steg 2 startar från den mindre ASP.NET-runtime-imagen och kopierar bara in den publicerade appen, så kompilator och källkod följer inte med. .dockerignore håller bin/, obj/, .env och .git/ utanför byggkontexten.

Imagen taggas v1 och pushas till mitt Container Registry. az container create startar en Container Instance med publik IP, DNS-namn och miljövariablerna CITY_NAME, NODE_URL och REGISTRY_URL. Vid start registrerar noden sig mot registret. Andra noder hämtar min adress därifrån och skickar paket till POST /paket; är destinationen Trollhättan sparar paketet, annars räknar Dijkstra ut nästa stad och ForwardAsync skickar det vidare.

2. NODE_URL-problemet

Noden måste skicka sin egen adress till registret när den startar. Den publika IP-adressen får containern först efter att den skapats, så för att skapa containern behöver jag IP:n, och för att få IP:n måste containern redan finnas. Dessutom delas en ny IP ut varje gång containern skapas om, så även en avläst IP slutar gälla.

Här fick jag tänka om: istället för att försöka få tag på IP-adressen bestämde jag adressen själv. Med --dns-name-label scantrack-trollhattan får containern namnet scantrack-trollhattan.swedencentral.azurecontainer.io, alltså mitt namn plus region plus en fast ändelse. Eftersom mönstret är känt i förväg kan jag skriva in hela adressen som NODE_URL i

samma `az container create` som skapar containern. Azure håller namnet pekande på containerns aktuella IP, så adressen fortsätter gälla oavsett omstarter. DNS-namn tillåter bara a-z, siffror och bindestreck, därför heter containern `scantrack-trollhattan` medan `CITY_NAME` är Trollhättan med ä för att matcha `cities.csv`.

3. Resonemang

COPY-ordningen och byggtider. Docker sparar varje instruktion i Dockerfilen som ett lager och återanvänder lagret så länge inget före det har ändrats. Kopieras all kod före `dotnet restore` ogiltigförklaras restore-lagret vid varje kodändring, och alla NuGet-paket laddas ner och verifieras på nytt. Kopieras bara `.csproj` först ligger restore-lagret kvar tills paketlistan faktiskt ändras. Konkret: mitt första bygge tog 37 sekunder. Efter en kodändring tog ombygget 7,3 sekunder, och utdatan visade `CACHED` på både `COPY`

`ScanTrackNode.csproj` och `RUN dotnet restore`; bara kodkopieringen och `dotnet publish` kördes om.

ACI framför App Service och AKS. Container Instances kör en enskild container med publik adress på under två minuter, debiteras per sekund och kräver ingen infrastruktur, vilket passar en nod som tas bort efter inlämning. App Service är byggt för webbappar och tar över fler plattformsbeslut (portar, skalning, deploy-slots) än den här noden behöver. Kubernetes Service ger självläkning, autoskalning och uppdateringar utan avbrott, men kräver ett kluster; för en container är det överdimensionerat. Med alla städer i drift och krav på tillgänglighet hade AKS varit rätt val.

Vad som saknas jämfört med produktion. Inloggningen mot registret sker med ACR:s admin-lösenord, som skickas i klartext i `deploy-kommandot`. I produktion skulle containern ha en managed identity med rollen `AcrPull`, och övriga hemligheter skulle ligga i Azure Key Vault istället för i miljövariabler. Det finns heller ingen health check: utan en `LivenessProbe` räknas containern som frisk så fort processen startat, och om appen hänger sig startas den inte om automatiskt.

Individuell reflektion. Svårast var `NODE_URL`-problemet, eftersom det inte gick att lösa med mer kod utan krävde att jag tänkte om kring var adressen kommer ifrån. Det jag förstår nu som jag inte förstod innan är hur mycket av arbetet som ligger i miljön runt koden: att en image är en ögonblicksbild som måste byggas och pushas om vid varje ändring, att ordningen i Dockerfilen avgör byggtiden, och att ett stabilt namn är viktigare än en IP-adress. Hade jag gjort om det hade jag använt `--dns-name-label` redan från den första deployen och satt upp `.dockerignore` innan det första bygget.

4. Bevis (skärmdumpar bifogas)

1. `GET /status` från ACI: stad: Trollhättan, url: `http://scantrack-trollhattan.swedencentral.azurecontainer.io:8080`.
2. `az container logs`: "Nod registrerad: Trollhättan → `http://scantrack-trollhattan...`", "Paket `PKG-CF9245` anlände till Trollhättan", "levererat till Trollhättan!" samt paketet `PKG-99832A` som anlände, routades och vidarebefordrades: "Vidarebefordrar `PKG-99832A`: Trollhättan → Borås", "skickat till Borås — svar 200".
3. `docker build` efter en kodändring med `CACHED` på restore-steget, 7,3 s mot 37 s.
4. `POST /forceheartbeat` som svarar "heartbeat skickat", och 429 vid nästa anrop inom tio minuter.